

Proyecto Semestral, Diseño Orientado a Objetos 2026-1

Grupo Número 3:

Benjamín Antonio Rivas Sepúlveda

Ignacio Moises Lama Méndez

Joaquín Enrique Jara Troncoso

Enunciado:

“Este sistema está diseñado para facilitar la organización de torneos deportivos o de juegos. Permitirá a un organizador definir las características del torneo, como el nombre, la disciplina (ej. fútbol, ajedrez, videojuegos), las fechas y un formato principal (como eliminatoria directa, eliminatorio doble, liga simple...). Se podrán inscribir participantes, ya sean jugadores individuales o equipos, almacenando información básica como nombres y datos de contacto. El sistema deberá ser capaz de generar un calendario de enfrentamientos o un bracket inicial basado en los inscritos y el formato. Durante el torneo, se registrarán los resultados de cada enfrentamiento, lo que actualizará automáticamente las posiciones, el avance en el bracket o las tablas de clasificación. Los usuarios podrán visualizar el estado actual del torneo, los próximos encuentros y las estadísticas generales.”

Diagrama de Caso de Uso:

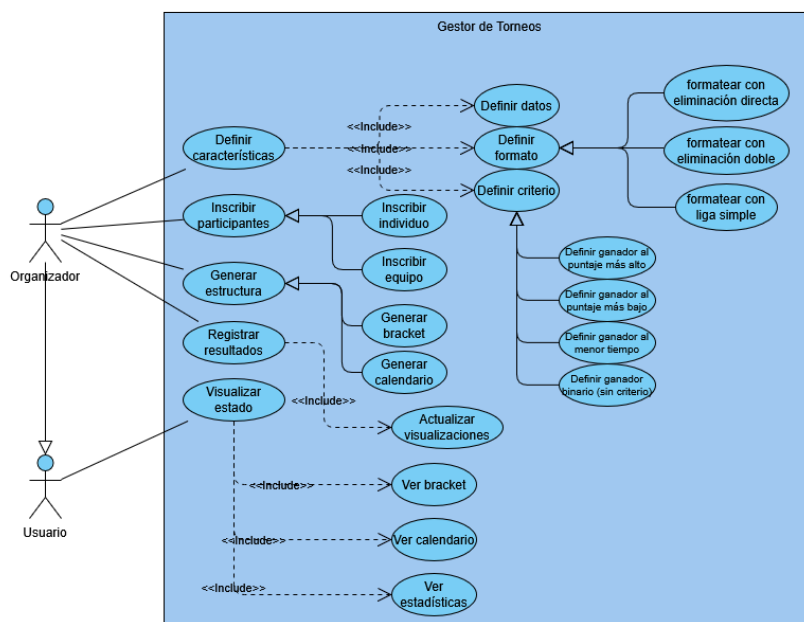
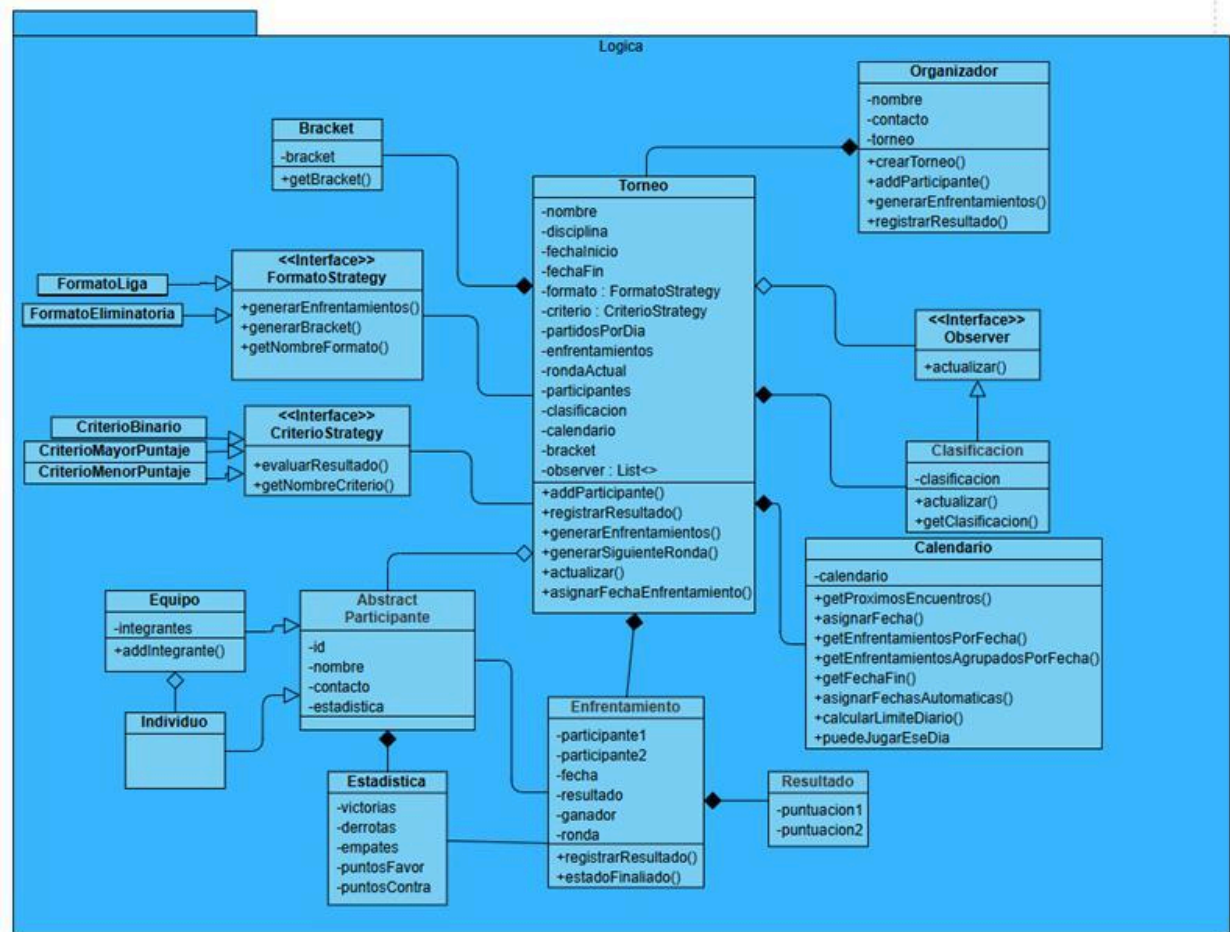


Diagrama de Clases UML:



Captura de pantalla de la interfaz:

TORNEO

Sin crear

DISCIPLINA

Sin definir

FORMATO

Sin definir

CRITERIO

Sin definir

FECHA FIN

Por calcular

PARTICIPANTES

0

ESTADO

Pendiente

GANADOR

Sin definir

Organizador

DATOS DEL TORNEO

Nombre del torneo:

Disciplina:

Fútbol

Formato:

Liga

Criterio ganador:

Mayor puntaje

Partidos por día:

1

Fecha inicio:

Elegir

Calendario mensual

Clasificación

Bracket

<

Julio 2026

>

LUN	MAR	MIE	JOE	VIE	SAB	DOM
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

ENCUENTROS DEL DÍA SELECCIONADO

No hay encuentros programados para este día

Marcar del encuentro

Seleccione un encuentro

Participante 1:

VS

Participante 2:

Registrar resultado

Seleccione un enfrentamiento

Patrones:

Para este proyecto, notamos que en múltiples ocasiones vamos a tener que recibir un mismo tipo de resultado a través de diferentes procesos, como decidir los enfrentamientos según diferentes formatos o elegir ganadores según diferentes criterios, así que decidimos implementar un patrón Strategy para estos dos elementos del torneo a través de las clases **FormatoStrategy** (interfaz), **FormatoLiga** y **FormatoEliminatoria** (algoritmos particulares) para los formatos de los enfrentamientos, y **CriterioStrategy** (interfaz), **CriterioMenorPuntaje**, **CriterioMayorPuntaje**, **CriterioBinario** (algoritmos particulares). Además, implementamos un patrón **Observer** a través de una interfaz del mismo nombre ya que la clase **Clasificación** requiere ser actualizada cada vez que haya un cambio en los participantes o resultados de un torneo.

Proceso:

Cuando comenzamos a conceptualizar el programa, decidimos que, si se van a admitir múltiples disciplinas, también el programa debería tener la capacidad de evaluar los ganadores según diferentes criterios según lo que determine el usuario, por lo que decidimos añadir la estructura para implementar esta modularidad a través de un patrón Strategy.

A lo largo del desarrollo notamos problemas menores con ciertas características planeadas inicialmente que más adelante notamos no eran adecuadas, como permitir que el usuario defina la “fechaFin” del Torneo o que la clase Calendario deba actualizarse junto a clasificación, por lo que pequeños detalles del producto final difieren frente a los diagramas, pero finalmente se llegó a un resultado funcional y un programa adecuado para el enunciado propuesto.